Fractus White Paper v2.0

A Continuous Thought Engine with Multi-Block Depth, Self-Modification, and Progressive Growth

Author: Philippe-Antoine Robert **Contact:** rpa.tu@proton.me **Date:** August 6, 2026 **Version:** 2.0 **Repository:** github.com/AFKmoney/fractus-test **Model Hub:** huggingface.co/thefinalboss/Fractus-1B

Abstract

I present Fractus v2.0 — a continuous cognitive agent architecture that departs fundamentally from the transformer paradigm. Unlike static models that map input to output in a single forward pass, Fractus is a **dynamical system** that maintains a persistent thought state, advances it tick by tick through a multi-block residual stack, and emits output only when it has something confident to say.

This version introduces three structural advances over the original:

1. **Multi-block depth** — the Continuous Thought Engine (CTE) now stacks N blocks, each with its own attention state (S,z), Kuramoto oscillator phases, and PhaseRoutedMoE. The thought flows through the stack as a residual stream, with per-block state carried continuously across chunk boundaries.

2. **Progressive growth** — instead of training a large model from scratch, Fractus grows palier by palier (width + depth + experts), inheriting previous weights via zero-padding. Each palier starts warm and converges faster.

3. **Runtime self-modification** — the model detects routing imbalance and grows new experts while it runs, with zero-init stability validated.

I also report **negative results** honestly: Expert Decoupled Training (EDT) and the Forward-Forward algorithm were both tested and refuted — their objectives are misaligned with the final cross-entropy loss. The only training method that works is standard gradient descent, but the architectural optimizations I describe (sparse low-rank MoE, head-partial training,

gradient accumulation, Kuramoto detachment) achieve **707 tokens/second on a consumer CPU** — a 177x improvement over the baseline.

---

1. Introduction

Contemporary large language models (GPT-4, Claude, Llama) share four fundamental limitations: they are static functions (one forward pass per output), stateless (no memory between conversations), generic (one monolithic network for all tasks), and centralized (training requires datacenter GPUs).

Fractus challenges each of these assumptions. The Continuous Thought Engine replaces the static function with a dynamical system. Persistent Memory gives the engine a cross-session memory bank. Expert Specialization forces each MoE expert to own a distinct skill domain. And the LazyStructuredSiren compression combined with progressive growth enables training on consumer hardware.

The question is not whether Fractus matches GPT-4 on benchmarks. It does not. The question is whether the paradigm of continuous, personal, decentralized AI is viable. This work demonstrates that it is — with measured, reproducible results.

---

2. Architecture

2.1 The Continuous Thought Engine (CTE)

The CTE is a dynamical system that maintains a persistent thought state `h ∈ R^{d_model}` and advances it tick by tick. The engine stacks `n_layers` blocks, each refining the thought:

```
h → [Block 0: norm → attn → +residual → norm → kuramoto → phases → norm →
  → [Block 1: ... ]
  → ...
  → [Block N: ... ] → thought_state = h_final
```

Each `CTEBlock` owns: - **FractalLinearAttention** (Katharopoulos 2020) — multi-level causal linear attention with a persistent state (S, z) that accumulates across ticks and chunk boundaries. - **Kuramoto oscillators** — a coupled dynamical system (low-rank RK4) that

acts as a "consciousness clock," producing phase vectors that route MoE experts. - **PhaseRoutedMoE** — a sparse mixture-of-experts with von Mises gate on Farey-distributed expert phases.

The thought state `h` is a **residual stream** — each block adds its transformation. The attention state (S, z) is **per-block** and **continuous across chunk boundaries** (verified: S grows monotonically across chunks, never reset).

2.2 PhaseRoutedMoE (Sparse, Low-Rank, Differentiable)

The MoE routes tokens via Kuramoto oscillator phases through a von Mises gate:

```
g_e = exp(κ · cos(θ_token − θ_expert)) / Σ_e' g_e'
```

Expert phases are drawn from the Farey sequence F_{2E}, providing E angles in [0, 2π) that are dense, non-collapsing, and deterministic. Only `top_k=2` experts are computed per token (gather-first sparse dispatch).

Low-rank experts: each expert weight matrix W is decomposed as `W = scale · U@Vᵀ` (rank r=64). The forward pass is two cheap matmuls that never materialize the full matrix. The sparse path gathers only the top-k experts' U/V factors, computing K experts instead of E. At 128 experts with top-k=2, this is 64x less compute.

Self-modification: `add_expert()` grows a new expert at runtime, placed near the dominant expert's phase (to capture overflow traffic), with zero-init (no forward perturbation). `maybe_grow()` triggers automatically when routing imbalance exceeds a threshold.

2.3 Multi-Block Depth

The original CTE (v1.0) had a single attention + Kuramoto + MoE block. This was the deepest limitation: depth 1 means the thought traverses one refinement then exits.

v2.0 introduces the `CTEBlock` abstraction. The engine stacks N blocks, each with independent state. The thought flows through all of them as a residual stream. This is the path to 1B+ parameters:

Config	d_model	blocks	experts/block	params
Palier 0	128	1	4	6.6M
Palier 1	256	2	8	~25M
Palier 2	512	4	16	~120M
Palier 3	768	8	32	~350M
Palier 4 (1B)	1280	16	128	~1B

2.4 Persistent Memory

The engine maintains a bank of memory vectors (d_model-dimensional, with context labels and importance scores) that survives across sessions. Memories are recalled via cosine similarity and injected into the thought state at 5% blend (continuous injection).

A **salience head** (Linear(d_model → 1)) learns to predict how much a memory injection will perturb the thought state — an intrinsic signal, not an external label. The system discovers its own sensitivity to memories.

2.5 Cognitive Modes

Cognitive modes emerge from the Kuramoto phase dynamics via unsupervised k-means clustering on phase features (synchronization degree r, mean phase, variance, per-oscillator sin/cos). No external labels — the modes are discovered from the structure of the phase space.

2.6 Self-Modification

Fractus is the only model that grows new capacity while it runs:

```
 engine.maybe_grow()
# → "[Fractus] Self-modified: grew expert in all 16 blocks (now 129 expert
```

The new expert is zero-initialized (scale=0 → output=0 → no gradient spike), placed near the dominant expert (captures overflow traffic), and warms up gradually via backprop. Validated: new expert receives 50% of traffic, loss stable post-grow.

2.7 Progressive Growth

Instead of training 1B from scratch (months on GPU), Fractus grows palier by palier. Each palier: 1. Inherits the previous model's weights via zero-padding (`fractus/grow.py`) 2. Old knowledge preserved (top-left block of every matrix) 3. New capacity starts neutral (zeros for weights, ones for LayerNorm gamma) 4. Trains briefly to adapt the new dimensions

This is how a brain develops: small at first, growing new capacity on top of existing knowledge.

---

3. Training

3.1 Online Training

The CTE trains online: one chunk (32 tokens) per forward, one backward per chunk. The thought state carries forward (detached — no BPTT). Each chunk's attention state (S,z) starts from the previous chunk's accumulated state — continuous thought.

3.2 Training Optimizations (Measured)

Optimization	What it does	Impact
Tied head	`output_head.weight = observe.weight`	Halves vocab params
Head-partial (`tick_chunk_train`)	Head on 1 position instead of C	32x less head FLOPs
Sparse MoE low-rank	Only top-k experts computed (gather-first)	64x at 128 experts
Gradient accumulation (accum=8)	8x fewer optimizer steps	16x fewer AdamW calls
Chunk_len=32	Better Python amortization	~1.1x
Detach Kuramoto	Phase computation in no_grad	

Optimization	What it does	Impact
		Removes backward through clock
bf16 AMP (GPU)	2x on all matmuls	GPU only

Combined measured result: 4 tok/s → **707 tok/s** on CPU (palier 0, d=128).

3.3 Profile Breakdown

Per-iteration cost at d=128, chunk_len=32:

Component	Time (ms)	% of forward
Attention (QKV + causal vectorized)	21.8	32%
Kuramoto (detached)	0.0	0%
MoE (4 experts, dense)	16.6	25%
Output head (1 position, tied)	12.6	19%
Embedding + norms	16.3	24%

At 128 experts, the sparse MoE path reduces MoE cost by 64x, making attention the dominant cost — as it should be for a reasoning architecture.

---

4. Negative Results (Honest)

4.1 EDT (Expert Decoupled Training) — Refuted

EDT claimed 189x training speedup by pre-training experts independently. I tested 5 variants (vanilla, denoise, identity, residual objectives + routing filter) on the 13M CTE:

Variant	Hold-out PPL	vs From-scratch
From-scratch	1309.7	—

Variant	Hold-out PPL	vs From-scratch
EDT next_hidden	1562.1	+19.3% worse
EDT denoise + routing filter	1555.1	+18.7% worse
EDT identity + routing filter	1556.8	+18.9% worse
EDT residual + routing filter	1573.7	+20.2% worse

Root cause: two structural defects. (1) The Phase-1 MSE objective (predict next hidden state) is misaligned with the Phase-3 CE objective (next-token prediction) — Pearson correlation never positive across 12 configs. (2) The Kuramoto router concentrates traffic on 2/4 experts, so half of pre-trained experts are never routed.

4.2 Forward-Forward (Hinton 2022) — Refuted for CTE

The Forward-Forward algorithm (local goodness signal, no global backprop) was adapted to the CTE. Result: NLL went UP (124 → 221). The goodness signal (sum of squared activations) is not aligned with cross-entropy. Local learning objectives cannot replace global backprop for this architecture.

4.3 What Works

Only standard gradient descent (CE + backprop) produces a model that learns. The optimizations described in §3.2 are the path to making this feasible.

---

5. Experimental Results

5.1 Progressive Growth (CPU)

Palier	d_model	blocks	params	tokens	loss	tok/s	time
0	128	1	6.6M	2M	32.5	725	46 min
1	256	2	25M	1.5M	27.1	395	63 min
2	512	4	120M	1M	27.1	192	87 min

Palier	d_model	blocks	params	tokens	loss	tok/s	time
3	768	8	350M	500k	23.0	122	68 min

Each palier starts warm (inherited weights) and converges. The corpus includes Fractus's own source code (palimpseste principle: the model contains its own description).

5.2 Self-Modification Stability

After runtime `add_expert()` at tick 500 (4→5 experts): - New expert receives 50% of routing traffic (placed near dominant) - Gradient norm stable (zero-init → no spike) - Loss post-grow: +24.6% (vs control no-grow: +67.6%) — growth helps stability

5.3 Continuous Thought Verification

Attention state (S,z) verified to grow monotonically across chunk boundaries in all three paths (tick, tick_chunk, tick_chunk_train). The thought is truly continuous.

5.4 Cross-Session Memory

Session 1: 4 memories captured and saved to disk. Session 2 (fresh engine): 4 memories loaded, thought state displaced by 100.16 units on first tick. Cross-session persistence verified.

---

6. Comparison with GPT and Claude

Property	GPT-4 / Claude	Fractus v2.0
Processing	Static (1 forward)	Continuous (ticks through N blocks)
Memory	Context window	Persistent bank + salience-gated injection
Skills	Generic monolith	Specialized MoE experts (128 per block)
Mental state	Stateless	Cognitive modes (unsupervised)
Generation	Token-by-token	Plan then fill + adaptive depth
Training	Datacenter GPUs	

Property	GPT-4 / Claude	Fractus v2.0
		Consumer CPU (progressive growth) + GPU for 1B
Deployment	Cloud API	Local device
User data	Sent to server	Stays local
Self-modification	None	Runtime expert growth
Depth scaling	Retrain from scratch	Progressive growth (warm start)

---

7. Limitations and Future Work

1. **Model quality**: The trained model at palier 3 (350M, 500k tokens) produces repetitive text. More data and training are needed for coherent generation. Chinchilla-optimal (940M tokens) requires ~5 days on GPU.
2. **Multi-block training**: The multi-block architecture is validated (gradient flows through all blocks, continuous thought works) but has not yet been trained at scale. Palier 4 (16 blocks, 128 experts, 1B params) awaits GPU compute.
3. **EDT and Forward-Forward**: Both refuted. Alternative training acceleration methods must align their objective with the final CE loss.
4. **Vocabulary**: The GPT-2 BPE vocab (50257) dominates parameters (81% at d=768). A reduced vocab (8k-16k) would cut head FLOPs by 3-6x.
5. **Corpus**: The quality corpus (20.5M tokens) includes Fractus's own source code but is far below Chinchilla scale for the larger paliers.

---

8. Conclusion

Fractus v2.0 demonstrates that a continuous, multi-block, self-modifying cognitive agent can be constructed and progressively trained on consumer hardware. The architecture —

CTEBlock stack with per-block continuous attention state, PhaseRoutedMoE with sparse low-rank experts, progressive growth via zero-padding, and runtime self-modification — is validated by 28 tests and measured benchmarks.

The negative results (EDT, Forward-Forward) are reported honestly. They do not weaken the architecture; they clarify what works (global backprop + architectural optimizations) and what does not (decoupled/local training).

The implications extend beyond performance metrics. If AI can be trained and deployed on any laptop, the centralization of intelligence by a handful of corporations is not inevitable. Fractus is a proof of concept for decentralized AI: intelligence that belongs to the user, runs on their hardware, remembers them, and grows.

This work is released as open source under the MIT license. All code, training scripts, datasets, and measured results are available at github.com/AFKmoney/fractus-test.

References

[1] Katharopoulos et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML. [2] Sitzmann et al. (2020). Implicit Neural Representations with Periodic Activation Functions (SIREN). NeurIPS. [3] Hinton, G. (2022). The Forward-Forward Algorithm: Some Preliminary Investigations. [4] Kuramoto, Y. (1984). Chemical Oscillations, Waves, and Turbulence. Springer. [5] Hinton, G. (2022). The Forward-Forward Algorithm. [6] Rahimi & Recht (2007). Random Features for Large-Scale Kernel Machines. NeurIPS. [7] Graves, A. (2016). Adaptive Computation Time for Recurrent Neural Networks. arXiv. [8] Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv.
